

CAIM Sessió 2: Programant amb Elasticsearch

Ferriol Falip Torras i Martí Puerta Juvés

17/10/2024

1 Aplicant diferents tokenizers i filtres

En aquesta primera part de la pràctica tenim com a objectiu observar les diferents maneres de tractar el text que ens ofereix Elasticsearch per tal d'obtenir les paraules a analitzar de manera més precisa. D'aquesta manera, portem a terme un anàlisi dels resultats obtinguts a l'usar cadascun d'aquests mètodes emprant-los amb un índex de novel·les com a dades a tractar.

En primer lloc, comparem el nombre de tokens obtinguts a l'utilitzar cadascun dels diferents tokenitzadors.

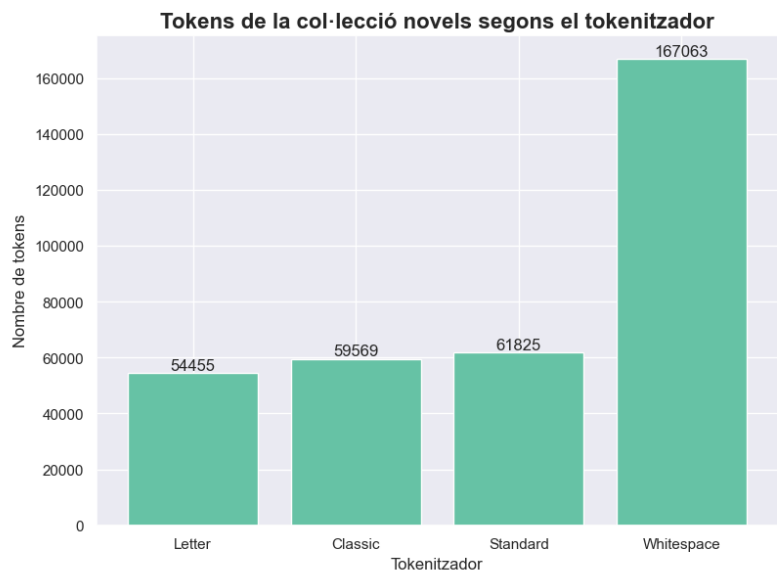


Figura 1: Visualització gràfica del nombre de tokens obtinguts de la col·lecció de novel·les segons el tokenitzador

Els resultats obtinguts mostren com *letter* és el tokenitzador més restrictiu i que menys tokens genera i *whitespace* el menys restrictiu.

A partir d'aquests resultats decidim utilitzar el tokenizer standard per comparar els diferents filtres, ja que *standard* no és tan restrictiu com *letter*, però redueixen en gran mesura el nombre de tokens igualment.

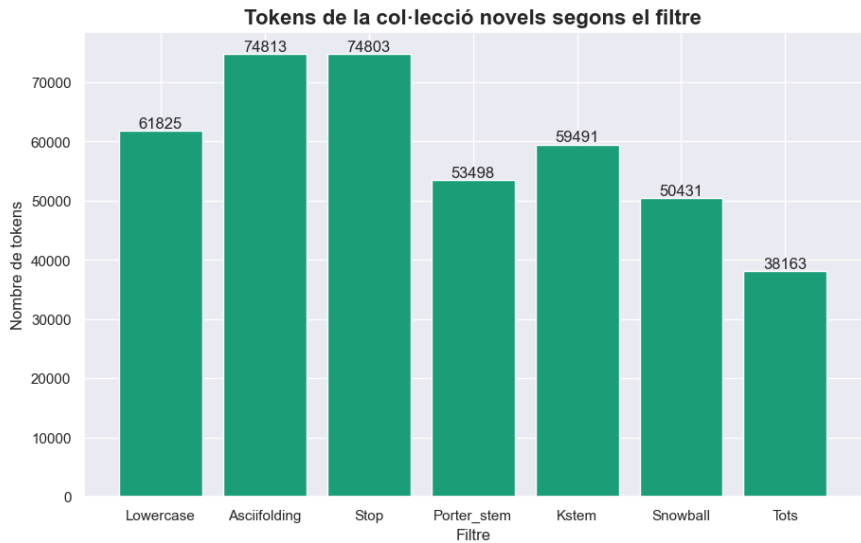


Figura 2: Visualització gràfica del nombre de tokens obtinguts de la col·lecció de novel·les segons el filtre

En conclusió, hem vist que la paraula més comuna en els textos de les novel·les és *"the"*, però a l'eliminar *stopwords* passa a ser *"we"*, la qual cosa ens a sorprèn lleugerament.

2 TF-IDF i similitud cosinus

2.1 TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) és una tècnica utilitzada en processament de llenguatge natural i recuperació d'informació per avaluar la importància (pes) d'una paraula dins d'un document en relació amb un conjunt de documents.

- TF (Freqüència de Termes): Mesura la freqüència d'una paraula en un document. Si $f_{d,i}$ és la freqüència del terme t_j al document d , llavors TF es calcula de la següent manera:

$$tf_{d,i} = \frac{f_{d,i}}{\max_j(f_{d,i})}$$

- IDF (Freqüència Inversa de Documents): Mesura la importància d'una paraula dins de tot el conjunt de documents. Si una paraula és comuna en molts documents, el seu valor IDF serà baix.

$$idf_i = \log_2 \frac{D}{df_i}$$

- TF-IDF: Multiplica el valor TF pel valor IDF per obtenir un valor que indica la importància d'una paraula en un document.

Aquest procés ens permet vectoritzar documents de text a partir dels termes i els seus pesos per tal de comparar la similitud entre documents de text mitjançant *cosine similarity*.

2.2 Similitud de cosinus

La cosine similarity (o similitud cosinus) és una mesura que s'utilitza per determinar la similitud entre dos vectors en un espai multidimensional. La similitud cosinus mesura l'angle cosinus entre dos vectors, que indica si apunten en la mateixa direcció o no. El valor resultant oscil·la entre -1 i 1 (En el nostre cas entre 0 i 1, ja que no hi ha freqüències de tokens negatives).

$$sim(a, b) = \frac{a \cdot b}{||a|| ||b||}$$

Per tal de fer aquest càlcul més eficient, recorrerem els 2 vectors de la mateixa manera que faríem en un algorisme de merge de llistes ordenades obtenint una complexitat de $\mathcal{O}(|a| + |b|)$. Això és possible, ja que els vectors estan ordenats alfabèticament.

```
def cosine_similarity(tw1, tw2):
    res = 0
    i = 0
    j = 0
    while i < len(tw1) and j < tw2:
        if tw1[i][0] == tw2[j][0]:
            res += tw1[i][1] * tw2[j][1]
            i += 1
            j += 1
        elif tw1[i][0] > tw2[j][0]:
            j += 1
        else:
            i += 1
    return res
```

Per comprobar el funcionament correcte d'aquesta implementació hem realitzat tres ràpides probes:

- Entre el document 1 i ell mateix hem obtingut una similitud de 1.
- Entre els document 4 i 3 hem obtingut una similitud de 0.035 (igual que a les transparències de l'assignatura).
- Entre els documents 1 i 7 hem obtingut una similitud de 0, és a dir, no tenen cap terme en comú.

Per tal de veure el funcionament de cosine similarity aplicat a casos més reals hem creat un script que donat un fitxer d'una categoria de la col·lecció *20_newsgroups* calcula la similitud amb 25 fitxers de cada grup de notícies. El fitxer que hem utilitzat pel següent gràfic és el fitxer *0001566* que pertany als grups *sci.electronics* i *cmp.sys.ibm.pc.hardware*.

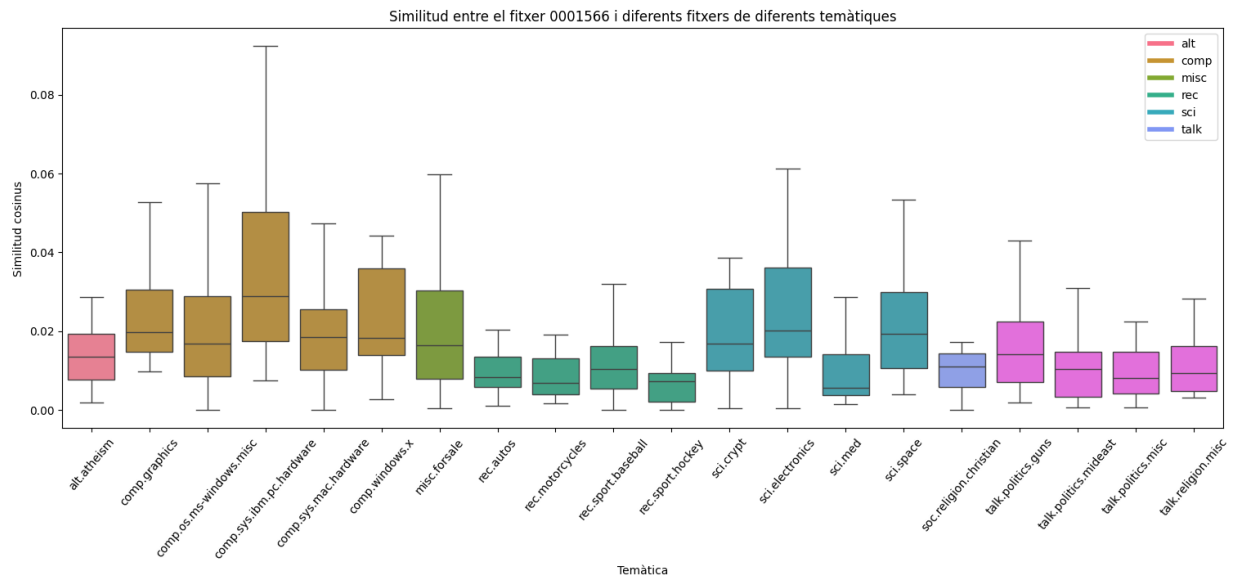


Figura 3: Visualització de la similitud del fitxer 0001566 amb diferents fitxer de diferents temàtiques de la col·lecció *20_newsgroups*

Com podem veure tot i que les similituds obtingudes són bastant baixes (inferiors a 0.1) tot i això si que es pot intuir una tendència i podem observar com les temàtiques recreatives o de política tenen en mitjana més baixa mentre que també tenen poca variància el que dona a entendre que realment els fitxers d'aquestes temàtiques no s'assemblen. Per altra banda, veiem com els fitxers d'electrònica i hardware són els que en mitjana s'assemblen més. Tenint en compte que el fitxer *0001566* és tant del grup d'electrònica com del de hardware per PC, podem concloure que els resultats obtinguts són prou fiables.

A més a més, hem comptat el temps que ha tardat el script a executar-se i hem obtingut un temps de 22.482 segons que tenint en compte que per cada dels 20 grups que hi ha hem analitzat 25 fitxers i que ho hem executat en una màquina virtual amb potència reduïda obtenim uns 45 ms per execució. Així doncs, creiem que experimentalment queda comprovat que el nostre mètode per calcular la similitud cosinus és prou eficient.

3 Valoració personal

Com a tancament de la pràctica, voldríem afegir que tot i entendre el codi que ens va ser proporcionat, vam començar la pràctica una mica perduts sobretot a l'hora d'implementar els codis demanats. Tot i això, en revisar la teoria i entendre correctament el que se'ns demanava de seguida vam ser capaços d'avançar i completar la pràctica. En general estem bastant satisfets amb els resultats obtinguts.